



Tecnológico de Monterrey

Campus Querétaro

Situación Problema
Equipo 6

Lucca Traslosheros Abascal A01713944
Oscar Lopez Cardoso A01713355

Análisis y diseño de algoritmos avanzados
Ramona Fuentes Valdéz
(Gpo 602)

23 de septiembre del 2026

1. Resumen ejecutivo

Este proyecto analiza transmisiones de texto para buscar códigos maliciosos conocidos, palíndromos largos y el substring más largo que comparten dos transmisiones. Está hecho en C++ y dividido en cuatro archivos (main.cpp, kmp.h, manacher.h y longestCommonSubstring.h) para separar las funciones de cada algoritmo. Se usan KMP y Manacher por su tiempo lineal, y programación dinámica para el substring común por su sencillez de implementación, aunque esta última requiere tiempo cuadrático respecto a las longitudes de las dos transmisiones.

2. Problemática

Cuando un dispositivo recibe una transmisión, esta trae una cabecera, datos y cola. Alguien que intercepta la transmisión puede insertar su propio código dentro de esa estructura sin que sea obvio a simple vista. El problema plantea tres preguntas concretas:

¿Podemos identificar código malicioso conocido dentro del flujo? Sí. Si conocemos una muestra del código malicioso (mcode), podemos buscarla en la transmisión con KMP y obtener la posición de su primera aparición.

¿Podemos localizar código «espejado» dentro de la transmisión? Sí. Bajo el supuesto del enunciado de que el código malicioso insertado es palindrómico, el palíndromo más largo es un bloque que merece revisión. Encontrarlo no demuestra por sí solo que sea malicioso ni señala con certeza dónde retoman los datos legítimos.

¿Podemos proponer un posible código compartido sin tener una muestra? Sí. El substring común más largo de las dos transmisiones proporciona un candidato para investigar. También podría ser contenido legítimo que ambas transmisiones comparten.

3. Objetivo general

Construir un programa que encuentre patrones conocidos, palíndromos y similitudes entre dos transmisiones mediante algoritmos apropiados.

Objetivos específicos

- Buscar los tres códigos con KMP en cada transmisión.
- Encontrar el palíndromo más largo de cada transmisión con Manacher.
- Encontrar el substring común más largo con programación dinámica.
- Integrar las tres operaciones en un programa ejecutable.

- Revisar el programa con los cinco archivos de entrada preparados.

4. Descripción del repositorio

- main.cpp: lee los archivos, llama a los algoritmos y muestra los resultados.
- kmp.h: búsqueda de patrones mediante KMP.
- manacher.h: búsqueda del palíndromo más largo.
- longestCommonSubstring.h: búsqueda del substring común más largo.
- transmission1.txt y transmission2.txt: transmisiones a analizar.
- mcode1.txt, mcode2.txt y mcode3.txt: posibles códigos maliciosos.

5. Funcionamiento de cada sección

5.1 Lectura de archivos. readFile() lee cada archivo carácter por carácter y descarta \n y \r. Por ello, las posiciones reportadas corresponden al contenido concatenado sin saltos de línea. El programa usa los cinco nombres de archivo fijos y termina con un mensaje de error si no puede abrir alguno.

5.2 Búsqueda de códigos maliciosos (KMP). kmpSearch recibe el texto y el patrón, construye una tabla de prefijos con prefixTable y recorre el texto aprovechando las coincidencias parciales. Devuelve la posición de la primera coincidencia en índice desde cero, o -1 si no hay ninguna. main.cpp suma uno antes de imprimir una posición encontrada. KMP evita reiniciar la búsqueda desde el siguiente carácter cada vez que ocurre una discrepancia.

5.3 Palíndromo más largo (Manacher). El texto se transforma agregando separadores entre caracteres para tratar de la misma manera los palíndromos de longitud par e impar. Después se calcula el radio del palíndromo centrado en cada posición y se devuelve el rango del más largo. En caso de empate, el código elige el que comienza antes. Manacher ejecuta esta búsqueda en tiempo lineal.

5.4 Substring común más largo (programación dinámica). Se compara cada carácter de la primera cadena con cada carácter de la segunda. Si coinciden, la longitud de la coincidencia se obtiene de la diagonal anterior; si no, permanece en cero. Se conserva el rango de la coincidencia más larga en la primera transmisión. En caso de empate, el código elige la que comienza antes. La implementación guarda solamente la fila anterior y la actual de la tabla.

6. Criterios de selección de los algoritmos

- **Eficiencia:** se emplean KMP y Manacher para resolver sus respectivas búsquedas en tiempo lineal; para el substring común se acepta el costo cuadrático de la programación dinámica.

- **Exactitud:** los algoritmos calculan coincidencias y rangos exactos, sujetos al contenido leído de los archivos.
- **Organización:** cada algoritmo se encuentra en su propio archivo .h.
- **Simplicidad:** la versión de programación dinámica permite seguir directamente cómo se construye una coincidencia común.

7. Complejidad de los algoritmos

Algoritmo	Tiempo	Espacio adicional
KMP	$O(n + m)$	$O(m)$
Manacher	$O(n)$	$O(n)$
Substring común (DP)	$O(n \times m)$	$O(m)$

En KMP, n es la longitud de la transmisión y m la del patrón. En la fila de programación dinámica, n y m son las longitudes de la primera y la segunda transmisión, respectivamente. Su espacio adicional es $O(m)$ porque se guardan dos filas de tamaño $m + 1$. El cálculo del substring común puede ser costoso para transmisiones grandes; una posible alternativa futura sería un *suffix automaton*.

7.1 Archivos de prueba. Se prepararon dos transmisiones (transmission1.txt y transmission2.txt) y tres códigos (mcode1.txt, mcode2.txt y mcode3.txt). Con estos archivos se pueden revisar las seis búsquedas de patrones y los rangos de los palíndromos y del substring común. No se realizaron pruebas aleatorias ni mediciones de rendimiento con archivos grandes.

8. Integración del sistema

main.cpp lee las dos transmisiones y los tres códigos, verifica que los archivos se puedan abrir, ejecuta las seis búsquedas de KMP, encuentra el palíndromo más largo de cada transmisión y calcula el substring común más largo. Luego imprime nueve líneas, en ese orden: seis resultados de búsqueda, dos rangos de palíndromos y un rango del substring común. Para una coincidencia imprime true seguido de un espacio y su posición; si no existe, imprime false. Cada rango se imprime como dos posiciones separadas por un espacio.

9. Resultados esperados

- Por cada transmisión y cada mcode: true y la posición donde empieza, o false.
- El rango del palíndromo más largo de cada transmisión.
- El rango del substring común más largo dentro de la primera transmisión.

Las posiciones que imprime el programa comienzan en 1. Para los cinco archivos de prueba preparados, los resultados esperados, en el orden requerido, son:

```
None  
false  
  
false  
  
true 18  
  
false  
  
false  
  
true 18  
  
15 25  
  
15 25  
  
1 28
```

10. Conclusiones

- KMP permite ubicar la primera aparición de un código conocido en una transmisión.
- Manacher encuentra el palíndromo más largo, que puede servir como candidato para revisar una posible alteración.
- El substring común más largo muestra una similitud entre las transmisiones, pero por sí solo no identifica su origen.
- Separar los algoritmos en archivos distintos facilita explicar el funcionamiento de cada sección.
- El cálculo del substring común es la parte con mayor costo asintótico: $O(n \times m)$. Para archivos grandes podría considerarse otro algoritmo.

11. Referencias bibliográficas (APA)

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Knuth, D. E., Morris, J. H., & Pratt, V. R. (1977). Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2), 323–350.

Manacher, G. (1975). A new linear-time “on-line” algorithm for finding the smallest initial palindrome of a string. *Journal of the ACM*, 22(3), 346–351.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Skiena, S. S. (2020). *The algorithm design manual* (3rd ed.). Springer.

Weiss, M. A. (2014). *Data structures and algorithm analysis in C++* (4th ed.). Pearson.

12. Cierre

El programa integra KMP, Manacher y programación dinámica para analizar patrones, palíndromos y similitudes en dos transmisiones de texto. Los resultados indican posiciones exactas en el contenido leído y ofrecen puntos de partida para revisar posibles alteraciones, sin afirmar que estas se puedan confirmar únicamente con los patrones encontrados.